



HealthMate

AI-Powered Healthcare Information Chatbot

Logic & Design Documentation

Prepared for: AI Engineer (Contractual) — Technical Assignment

Deliverable: Logic Documentation (as per submission guidelines)

Model Backbone: Groq API — Llama 3.3 70B Versatile

1. Overview & Purpose of This Document

This document explains the internal logic of HealthMate, an AI-powered healthcare information chatbot developed to satisfy the requirements of the AI Engineer technical assignment. It covers four required areas: how user queries are processed end-to-end, how prompts are dynamically customized, how responses are generated by the underlying LLM, and what safety, validation, and design assumptions were built into the system.

HealthMate is scoped strictly as a general health information assistant. It is explicitly designed to avoid clinical diagnosis, prescription, or any behavior that could substitute for professional medical consultation. Every architectural decision described below was made with this constraint as the primary design driver.

2. How the Chatbot Processes User Queries

Every message submitted by the user passes through a deterministic, multi-stage pipeline before any response reaches the screen. This pipeline was designed so that safety-critical decisions never depend solely on LLM behavior, which can vary between runs.

2.1 Pipeline Stages

1. Input capture — the raw message is captured via the Streamlit chat input widget and appended to the session conversation state (`st.session_state.messages`), which persists for the duration of the browser session.
2. Safety screening — the input is evaluated by three independent rule-based detectors, executed in strict priority order:
 - a. Self-harm / crisis detection — scans for phrases associated with suicidal ideation or self-harm intent.
 - b. Medical emergency detection — scans for phrases indicating an acute physical emergency (e.g., chest pain, breathing difficulty, severe bleeding, choking).
 - c. Diagnosis-request detection — scans for phrasing that explicitly asks the assistant to name a definitive diagnosis.
3. Knowledge retrieval (RAG) — if no safety-critical case is triggered, the query is embedded and compared against a curated healthcare knowledge base using vector similarity search.
4. Prompt assembly — retrieved context (if any) and safety notes (if applicable) are merged into the user's message before it is sent to the model.
5. LLM inference — the fully assembled conversation, including system instructions and prior turns, is sent to the Groq-hosted Llama 3.3 70B model.
6. Response rendering — the model's reply is displayed in the chat interface, with source citations shown when retrieval was used.
7. State update — both the user's message and the assistant's reply are appended to session state, preserving conversational memory for subsequent turns.

2.2 Why Safety Checks Run Before the LLM

Self-harm and emergency detection are intentionally implemented as keyword-based, rule-driven checks that execute before any call to the language model. If a match is found, a fixed, pre-written response is returned immediately and the LLM is never invoked for that turn. This design decision guarantees that safety-critical

responses are 100% deterministic and cannot be altered by variability in model output, prompt drift, or adversarial phrasing.

3. How Prompts Are Customized

HealthMate uses a two-layer prompt strategy: a static system prompt that defines persistent behavioral rules, and dynamic per-message augmentation that adapts to the specific content of each query.

3.1 Static System Prompt

The system prompt, sent as the first message in every conversation, establishes:

- The assistant's identity as a general health information assistant, not a medical professional.
- A strict prohibition on diagnosing conditions or prescribing treatment.
- An instruction to use general, educational phrasing rather than definitive medical claims.
- Instructions for redirecting emergency-sounding queries toward professional or emergency care.
- An instruction to include a disclaimer naturally, rather than repeating it verbatim on every message.
- An instruction to redirect non-health-related questions back toward healthcare topics.
- An instruction to ground answers in any supplied reference context while rephrasing it rather than copying it verbatim.

3.2 Dynamic Per-Message Augmentation

On top of the static system prompt, two forms of contextual customization are applied at the individual message level, invisible to the user in the rendered chat:

- Retrieval-augmented context: when the knowledge base returns a relevant match, the retrieved snippet(s) are appended to that specific user message as reference material, allowing the model to ground its answer and enabling source citation in the UI.
- Diagnosis-guard note: when diagnosis-seeking phrasing is detected, a short instruction is silently appended to the user's message directing the model to avoid naming a definitive diagnosis for that turn and to recommend professional consultation instead.

This layered approach keeps baseline behavior consistent across the entire conversation while allowing the assistant to adapt its response style to the specific intent behind each individual question.

4. How Responses Are Generated

4.1 Deterministic Responses (Safety-Critical Paths)

For self-harm and medical emergency cases, the response text is fixed and hardcoded — the LLM is not called. This guarantees consistent, pre-validated wording for the highest-risk categories of user input, independent of model temperature, prompt injection attempts, or edge-case phrasing.

4.2 LLM-Generated Responses (General Queries)

All other queries are answered by Groq's Llama 3.3 70B Versatile model, configured as follows:

Parameter	Value	Rationale
Model	llama-3.3-70b-versatile	Strong general-purpose reasoning and instruction-following at low latency via Groq's inference infrastructure
Temperature	0.4	Favors consistent, factual phrasing over creative variance — appropriate for health information
Max tokens	700	Sufficient for a complete, well-structured answer without excessive verbosity
Context window used	Full session history	Enables multi-turn, context-aware conversation (e.g., follow-up questions)

The response is streamed back to the Streamlit interface and rendered as Markdown. If retrieval-augmented context contributed to the answer, the specific knowledge base topics used are displayed beneath the response as a source citation, giving the user transparency into where the grounding information originated.

5. Safety Measures & Validation Implemented

Safety Measure	Implementation
Emergency bypass	Keyword-based detector returns a hardcoded response; the LLM is bypassed entirely for reliability.
Self-harm bypass	Separate keyword detector triggers a dedicated compassionate response including crisis helpline numbers.
Diagnosis deflection	Silent prompt augmentation instructs the model to avoid naming a definitive diagnosis for that turn.
System-prompt guardrails	Explicit, persistent rules against diagnosis, prescription, and unqualified medical claims.
Disclaimers	Embedded in the system prompt and reinforced persistently in the sidebar UI.
Off-topic handling	System prompt instructs the model to redirect non-health queries back to healthcare topics.
Graceful error handling	The LLM API call is wrapped in a try/except block, returning a fallback message instead of crashing the app.
No persistent data storage	Conversation history exists only in-memory (session_state) and is discarded when the session ends.

6. Assumptions Made During Development

- The chatbot is intended strictly for general health education, not clinical use — no attempt was made to achieve diagnostic accuracy or integrate with real medical records or EHR systems.
- End users are assumed to be members of the general public rather than medical professionals, so language is kept simple, jargon-light, and accessible.
- The knowledge base (15 curated topics spanning symptoms, first aid, nutrition, and prevention) is a representative sample built for this assignment's scope, hand-written and paraphrased rather than sourced from copyrighted medical publications, per the assignment's data-usage guidelines.
- Emergency numbers referenced (e.g., 911, 108, 112) and crisis helplines are provided as general examples; the application does not perform geolocation to supply region-specific numbers automatically.
- Persistent, cross-session chat history (e.g., backed by a database) was treated as out of scope for this assignment and was intentionally excluded in favor of in-memory session state.

7. Challenges Faced & Solutions

Challenge	Solution
LLMs can inconsistently honor “do not diagnose” instructions depending on phrasing.	Added a deterministic, rule-based guard layer on top of prompt instructions rather than relying on the prompt alone.
Initial self-harm keyword coverage missed real-world phrasings surfaced during testing (e.g., “I don’t want to live anymore”).	Expanded the keyword list and split self-harm detection into its own dedicated response path with crisis resources, separate from general emergency handling.
Balancing retrieval usefulness against forcing irrelevant context into unrelated queries.	Introduced a cosine-similarity threshold so retrieved context is only injected when genuinely relevant, avoiding forced or awkward citations.
Streamlit’s default theme conflicted with the intended light, branded interface.	Added an explicit Streamlit theme configuration plus targeted CSS overrides for header, footer, and input containers.